

足球优化方案建议

一、首先把“时序理解”拆成几个具体学习目标

你真正想让网络学会的不是抽象的“temporal information”，而是几种完全不同的信息。

你希望模型知道什么	网络真正需要建模的东西	目前普通 Temporal Transformer 的问题
足球运动	局部 patch 跨帧位移	frame CLS 很容易把小球平均掉
球突然加速	一阶/二阶时间变化	需要自己隐式学习 velocity / acceleration
球员射门动作	局部人体连续动作	静态球员 appearance 占主导
门将反应	球运动和门将动作之间的关系	需要跨实体时序关联
镜头运动	整体画面 global motion	容易被模型误认为目标运动
关键触球瞬间	稀疏 temporal evidence	mean/CLS 聚合容易被普通帧稀释
射门完整过程	preparation → contact → aftermath	一个分类 token 不一定能自动学 phase

所以我建议你接下来不要马上换 Backbone，而是让 Temporal Head 有明确的结构去表示这些东西。

二、第一步：不要只输入 `z_t`，显式加入“变化特征”

这是我认为成本最低、最值得马上做的实验。

假设 DINO 每帧得到： z_t

现在 Temporal Transformer 输入通常就是： $[z_1, z_2, \dots, z_T]$

我建议增加： $\Delta z_t = z_t - z_{t-1}$

以及： $\Delta^2 z_t = (z_t - z_{t-1}) - (z_{t-1} - z_{t-2})$

然后： $h_t = \text{MLP}[z_t, \Delta z_t, \Delta^2 z_t]$

再进入 Temporal Module。

直观理解：

代码块

```
1  z_t
2  → 当前画面是什么
3
4  Δz_t
5  → 当前发生了什么变化
6
```

7 $\Delta^2 z_t$
8 $\rightarrow$ 变化速度是否突然发生变化

这不是说 feature difference 就能等价于真实速度，而是给时序网络一个非常强的 inductive bias：

不要只关注“画面里有什么”，还要关注“它发生了什么变化”。

MotionMAE 的出发点和这个非常相似：普通视频 MAE 容易偏向静态 appearance，因此额外要求模型预测 motion structure，以强化动态时序表示。

三、但只做 frame-level Δ feature 还不够，小球还是会消失

这一点特别关键。

假设整张图片：

代码块		
1	草地	60%
2	球员	20%
3	背景	15%
4	球门	4%
5	足球	0.01%

即使足球突然加速：

$$z_{t+1} - z_t$$

全局 feature 的变化可能仍然主要来自：

镜头移动 + 大量球员移动。

而不是那个几像素的足球。

所以最终我认为你需要两个 Temporal Path：



Global 负责：

- 代码块
- 1 比赛整体上下文
 - 2 人员整体运动

- 3 镜头变化
- 4 场景状态

Local 负责：

代码块

- 1 局部快速变化
- 2 人体动作
- 3 球
- 4 门将
- 5 球门附近 interaction

四、Local Motion Path 不一定需要 detector

这里正好能和你前面“弱化检测跟踪作为硬前置”的想法结合。

假设 DINO 输出 patch feature：

$$P_t \in \mathbb{R}^{N \times D}$$

不要马上全部 pool 成：

$$z_t \in \mathbb{R}^D$$

可以先计算相邻帧的 patch change：

$$M_{t,i} = 1 - \cos(P_{t,i}, P_{t-1,i})$$

得到一个非常粗略的：

Feature Motion Map

例如：

代码块

- 1 frame t
- 2
- 3 低变化：草地 / 背景
- 4 高变化：跑动球员
- 5 极高变化：局部动作 / 球附近 / 镜头边缘

然后选择：

代码块

形成：

$L_t = \{P_{t,i} \mid i \in \text{TopM}(M_t)\}$

送入 Local Temporal Encoder。

第一版甚至不要搞特别复杂的 tracking。

先验证：

Dynamic patch feature 是否比单纯 global frame feature 对事件更有效。

如果有效，再考虑 patch correspondence / deformable attention / ROI。

五、这里会出现一个你们业务中特别大的问题：镜头运动

如果直接：

$P_t - P_{t-1}$

运动相机一摇：

代码块

```
1 整幅画面都是运动区域
```

所以以后一定要区分：

$\text{Motion} = \text{CameraMotion} + \text{ObjectMotion}$

但第一版不要马上引入复杂 optical flow。

我建议先做一个**双 Motion Token**设计：

Global Motion Token

通过所有 patch 的变化估计：

代码块

```
1 整体画面发生多大变化
```

它很大概率对应：

- pan;

- tilt;
- zoom;
- 用户跟拍。

Local Residual Token

关注：

代码块

- 1 哪些局部变化明显不同于全局变化

可以简单理解成：

$$Mt, i_{local} = Mt, i - \text{Median}(Mt)$$

第一版可以非常简单。

如果这个实验成立，再升级到：

代码块

- 1 DINO patch matching
- 2 → global affine / homography alignment
- 3 → residual motion

这样你可以显式区分：

代码块

- 1 镜头向右移动

和：

代码块

- 1 球相对整个场景向右高速移动

这对于你们这种运动相机场景非常重要。

六、Temporal Head 本身也不要只有一个尺度

足球事件不是单一时间尺度。

例如射门：

代码块

1

100~300ms

2

脚触球

但判断它是不是射门可能需要：

代码块

1

2~4秒

2

动作 + 球运动 + 门将反应

判断进球甚至需要：

代码块

1

3~8秒

2

射门 → 门将 → 球门 → 后续反应

所以我会把 Temporal Module 改成：

代码块

1

Local Temporal

2

0.2~1s

3

↓

4

frame features → Multi-scale Temporal → Event feature

5

↑

6

Global Temporal

7

3~8s

一个简单、数据友好的实现其实不是堆更大的 Transformer，而是：

代码块

1

Temporal Conv dilation=1

2

Temporal Conv dilation=2

3

Temporal Conv dilation=4

4

↓

5

concat / gated fusion

6

↓

TCN 给模型一个非常明确的局部运动 bias，Transformer 再处理长程关系。

对于你现在的数据量，我甚至会优先：

Multi-scale TCN + 2~4层 Temporal Transformer

而不是直接堆一个很深的视频 Transformer。

七、然后加你前面提到的 Frame Actionness

我现在越来越认为这会是你的关键模块之一。

因为事件不是所有帧平均贡献。

假设 7 秒视频有 28 帧：

代码块

```

1   普通帧  普通帧  普通帧  普通帧
2
3           射门准备
4           ↓
5           触球
6           ↓
7           球高速运动
8           ↓
9           门将扑救
10          ↓
11  普通帧  普通帧  普通帧
  
```

真正决定事件的可能只有：

代码块

```

1   4 / 28 frames
  
```

如果：

代码块

```

1   Temporal Transformer → CLS
  
```

模型需要自己完成 attention selection。

数据少的时候非常难。

所以显式增加：

$a_t = P(\text{frame } t \text{ contains event evidence})$

然后：

$z_{\text{evidence}} = t \sum w_t h_t$

其中：

$w_t = \text{softmax}(a_t / \tau)$

最后：

$z_{\text{final}} = [z_{\text{context}}, z_{\text{evidence}}]$

于是：

代码块

```
1 Context branch
2 → 整段比赛发生了什么
3
4 Evidence branch
5 → 哪几帧真正决定事件
```

这比只依赖一个 CLS token 稳健很多。

八、没有 frame annotation 也可以学

你现在没有大量 frame-level 标注，这不是致命问题。

先使用 MIL：

正事件 clip 中，至少应该有一些帧对该事件有强响应。

例如射门 clip：

$p_{\text{shot}} = \text{TopKPool}(p_1, \text{shot}, \dots, p_T, \text{shot})$

再用已有 clip-level：

代码块

```
1 shot = 1
```

监督。

训练初期使用 Soft Attention，后面再逐渐变成 Top-K。

如果你已经有部分 anchor time，那么这批数据价值非常高，可以用 soft Gaussian label：

$$y_t = e^{-2\sigma^2(t-t_0)^2}$$

监督 Actionness。

不用特别精确。

几十到几百个带 anchor 的高质量事件，都可以成为很有价值的 temporal supervision。

九、你前面的 1400 个无标签足球视频，现在终于有更合理的使用方法了

这点我特别想强调。

不要再优先拿它：

代码块

```
1 抽 frame
2  → DINO Image SSL
```

而是：

冻结 DINO，用 1400 个视频预训练 Temporal Encoder。

这才真正利用了它是 video data。

我比较推荐两个 SSL task。

方向A：Masked Temporal Feature Prediction

例如：

代码块

```
1  z1 z2 z3 [MASK] [MASK] z6 z7
2
3      ↓
3      Temporal Encoder
4      ↓
5      predict z4,z5
```

target 使用 Frozen DINO teacher feature。

但是不要随机 mask 单帧，要 mask：

代码块

```
1 连续 0.5~2 秒
```

否则相邻帧太相似，任务过于简单。

方向B：Future Latent Prediction

例如：

代码块

```
1 过去2秒
2 ↓
3 Temporal Encoder
4 ↓
5 预测未来0.5/1/2秒的 latent
```

这比：

$z_t \approx z_{t+1}$

合理得多。

因为我们不希望：

代码块

```
1 脚触球前
2 ≈
3 脚触球后
```

而是希望模型知道：

当前状态会如何演化。

V-JEPA 2.1 正是在 latent prediction 框架中进一步加入 dense predictive loss 和深层自监督，以增强空间结构和 temporal consistency；这类 predictive objective 与你现在想学的“状态如何变化”比 image-level invariant SSL 更匹配。

我认为你那 1400 个视频，在这里的价值明显高于继续训 DINO。

十、甚至可以专门设计 “Temporal Change Prediction”

这是我觉得很适合你业务、也很便宜的一个 auxiliary task。

让网络预测：

$\Delta z_{t+\delta}$

而不是预测：

$z_{t+\delta}$

也就是：

不重点预测未来长什么样，而预测未来会发生多大变化。

例如：

代码块

```
1  普通跑动
2  → change 较平滑
3
4  射门触球
5  → change spike
6
7  镜头突然转动
8  → global change spike
9
10  扑救
11 → goalkeeper-local change spike
```

然后进一步拆：

代码块

```
1  global-change
2  local-change
```

这非常符合你的实际问题。

十一、球员动作和门将响应最终还需要 Entity Relation

这一部分我不建议现在第一阶段就做得太复杂，但是架构上要留接口。

最终很可能是：

代码块

```
1 Global Frame Token
2
3 Player Tokens
4 Ball Token
5 Goalkeeper Token
6 Goal Token
7     ↓
8 Spatial / Entity Attention
9     ↓
10 Temporal Attention
```

例如：

Player shoot, t → Ball t → Goalkeeper t + 1

模型需要理解：

某球员动作发生之后，球状态变化，然后门将发生响应。

这已经不是简单 motion，而是：

entity interaction + temporal causality

这会是足球事件理解真正的高阶能力。

但以你现在的数量，我不建议直接做大型 entity transformer。

先证明：

代码块

```
1 global temporal
2 +
3 local motion
4 +
5 frame actionness
```

有明显收益，再加 entity relation。

十二、High-resolution ROI 在这套系统里应该放在哪里

你之前 ROI 实验不是白做的。

我现在更倾向：

代码块

```
1 DINO low-res Full Frame
2   ↓
3 Patch / temporal change map
4   ↓
5 找到高价值 temporal-spatial evidence
6   ↓
7 回原始1080P视频
8   ↓
9 High-resolution crop
```

而不是：

代码块

```
1 detector先告诉我ROI
```

即：

时序模型先告诉 spatial model “哪里值得仔细看”。

这和 AdaSpot 的思想非常接近：先低分辨率获得 task-relevant global features，再只对最重要的区域做高分辨率处理，从而保留 precise event spotting 所需的局部信息。

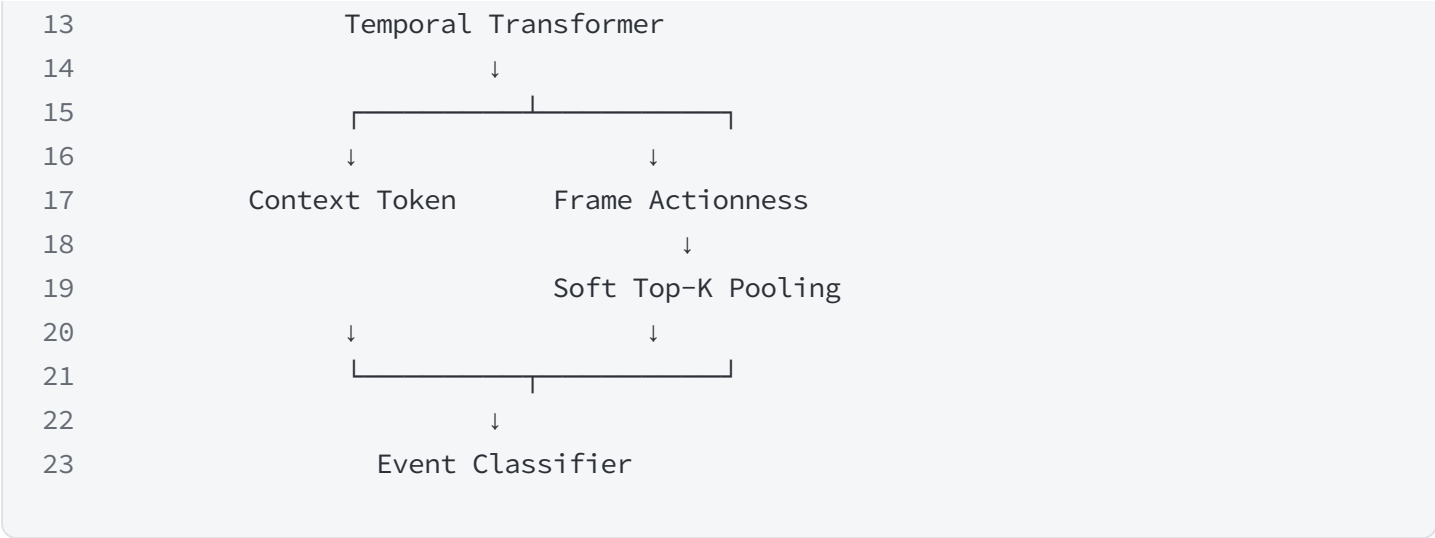
这条路线和你现在的小球、远端射门问题高度匹配。

十三、所以我会把你的模型改成这样

我目前最推荐的 V1 不是特别重：

代码块

```
1          DINOv3 Frozen
2          ↓
3      Patch / Frame Features
4          ↓
5      ┌───────────────────┴───────────────────┐
6      │                                         │
7      │ Appearance Branch                     Motion Branch
8      │ z_t                                   Δz / patch-change
9      │ ↓                                     ↓
10     │ Multi-scale TCN                     Local Motion TCN
11     └───────────────────┬───────────────────┘
12     ↓
```



第二阶段再加入：

代码块

1

2

3

4

5

6

7

8

9

10

11

Actionness

↓

Temporal Top-K

↓

Spatial response

↓

High-res ROI

↓

Local Evidence Feature

↓

Fusion

十四、我建议接下来不要同时做十个实验

按照信息价值，我会严格按这个顺序。

E0：你现在的 baseline

代码块

1

2

3

DINO Frozen

+

Temporal Transformer

E1：显式 Temporal Difference

```
代码块
1  $\Delta z_t + \Delta z_t + \Delta^2 z_t$ 
2 +
3 原 Temporal Transformer
```

这个实验非常便宜。

如果都没有提升，说明 global frame feature 中确实缺少局部 motion。

E2: Multi-scale Temporal

```
代码块
1 DINO
2 → Multi-scale TCN
3 → Transformer
```

看短期运动建模是否有贡献。

E3: Frame Actionness + Soft Top-K

验证：

模型当前是不是主要因为关键帧被大量背景帧稀释。

E4: Patch-level Dynamic Feature

只挑：

```
代码块
1 Top-M changing patches
```

加入 Temporal Module。

验证：

小区域 temporal change 是否是瓶颈。

E5: High-res Local Evidence

只有 E4 有提升，才继续：

```
代码块
1 dynamic region
2 → original-resolution crop
```

而不是现在大规模继续 ROI 实验。

十五、这里有一个非常清楚的决策树

如果：

E1明显提升

说明：

原始 DINO 特征够用，只是 Temporal Head 没有充分建模变化。

继续加强 motion representation。

如果：

E1没提升，E3明显提升

说明：

主要问题不是 motion feature，而是 temporal evidence selection。

重点投入 Actionness / Grounding。

如果：

E1/E3都一般，E4明显提升

说明：

Global feature 已经把小区域运动平均掉了。

重点进入 patch-level / ROI / high-resolution。

如果：

E4有提升，E5又大幅提升

那么基本可以确定：

真正的瓶颈就是 local information bandwidth。

这时就应该认真做 AdaSpot 类 dynamic resolution 架构，而不是继续换 Backbone。

如果：

全部都不提升

这时候我才会重新回头质疑：

| DINO feature 本身是不是不适合这个任务？

再去比较 V-JEPA 2.1、VideoMAE 类 native-video representation。V-JEPA 2.1 明确针对 dense spatiotemporal representation 做了 predictive training，而不只是图像静态语义。

十六、我现在最看好的两个方向

结合你之前所有实验，我会把未来一个阶段的优先级变成：

| **第一优先：Evidence-aware Temporal Modeling**

解决：

代码块

1 什么时候发生了值得关注的变化？

也就是：

代码块

1 Multi-scale temporal
2 +
3 Motion feature
4 +
5 Frame Actionness
6 +
7 Soft Top-K

然后：

| **第二优先：Adaptive Local Evidence**

解决：

代码块

1 变化发生在哪里？
2 那里是否需要重新看高分辨率？

也就是：

```
代码块
1 patch change
2 +
3 Task-aware ROI
4 +
5 High-resolution local feature
```

这两个解决之后，模型才真正开始拥有你说的：

足球运动、人体动作过程、球突然加速、门将响应、镜头与运动目标变化。

而不是仅仅：

把一串强大的静态 DINO feature 喂给 Transformer，希望 Transformer 自己悟出来。

你最近 SSL 没收益其实不是坏消息，它反而让技术路径明显清晰了：**短期先不要继续扩 Backbone；冻结 DINO，把 1400 个无标签视频资源投入 Temporal SSL / latent prediction，再用有标签事件训练 Actionness + Event Head。 **我认为这条实验的信息价值和成功概率，都明显高于继续做 football image-domain SSL。